PERTEMUAN 2 - mus

▼

📂 Pembuatan Activity dan Components

Pada bagian ini kita akan membuat sebuah Activity baru untuk mulai berlatih menggunakan komponen dasar Android seperti `EditText`, `TextView` dan `Button` di dalam sebuah `ConstraintLayout`

📱 Activity

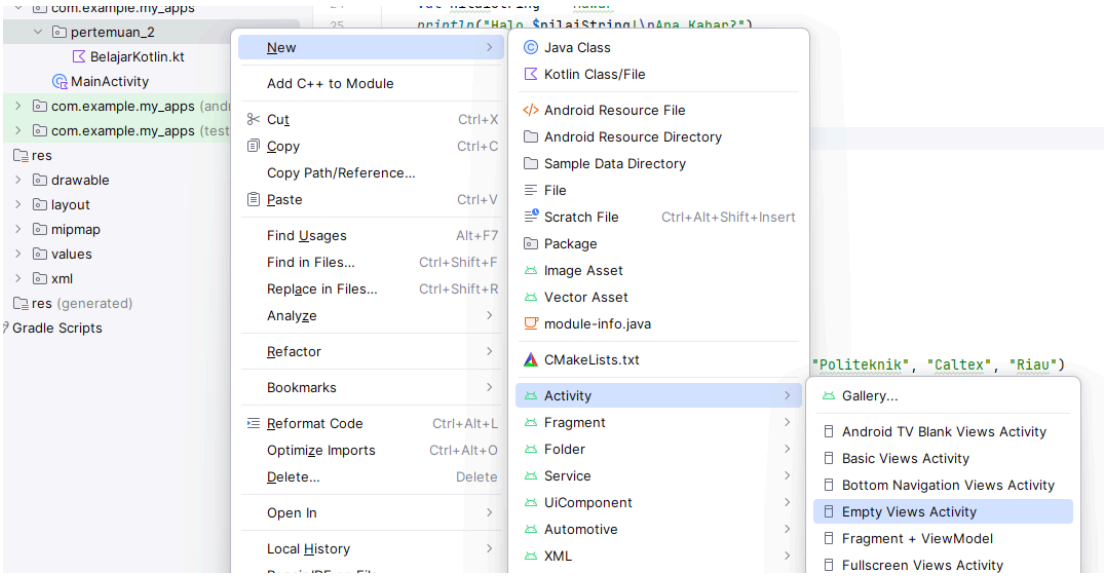
- **Activity = 1 layar** dalam aplikasi Android.
- Menampilkan 1 antarmuka (UI) tertentu.
- Bisa berpindah ke Activity lain.
- Contoh: Login, Dashboard, Setting.

🎨 Layout

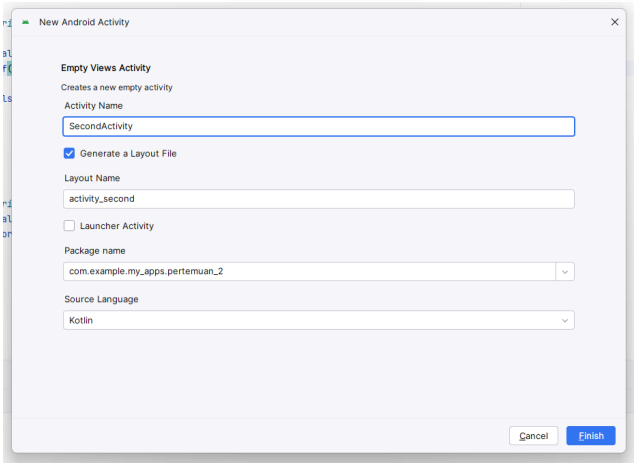
- Layout = file XML untuk mendesain tampilan Activity.
- Berisi komponen UI: `TextView`, `EditText`, `Button`, `ImageView`, dll.
- ConstraintLayout = layout fleksibel untuk mengatur posisi komponen.

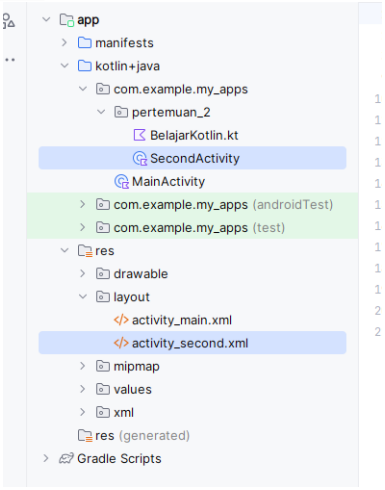
Silahkan ikuti beberapa langkah berikut:

1. Pada *package* `pertemuan_2` klik kanan → **New** → **Activity** → **Empty Activity**.

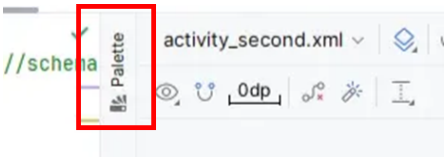


2. Isi ‘Activity Name’, dengan `SecondActivity` dan isian lainnya akan terisi secara otomatis, lalu klik **Finish** maka 2 file baru akan dibuat secara otomatis yaitu `SecondActivity` dan `activity_second.xml`

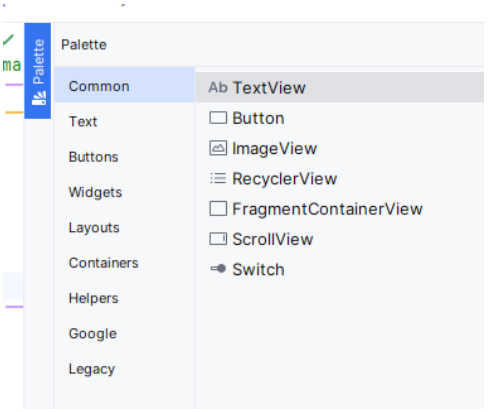




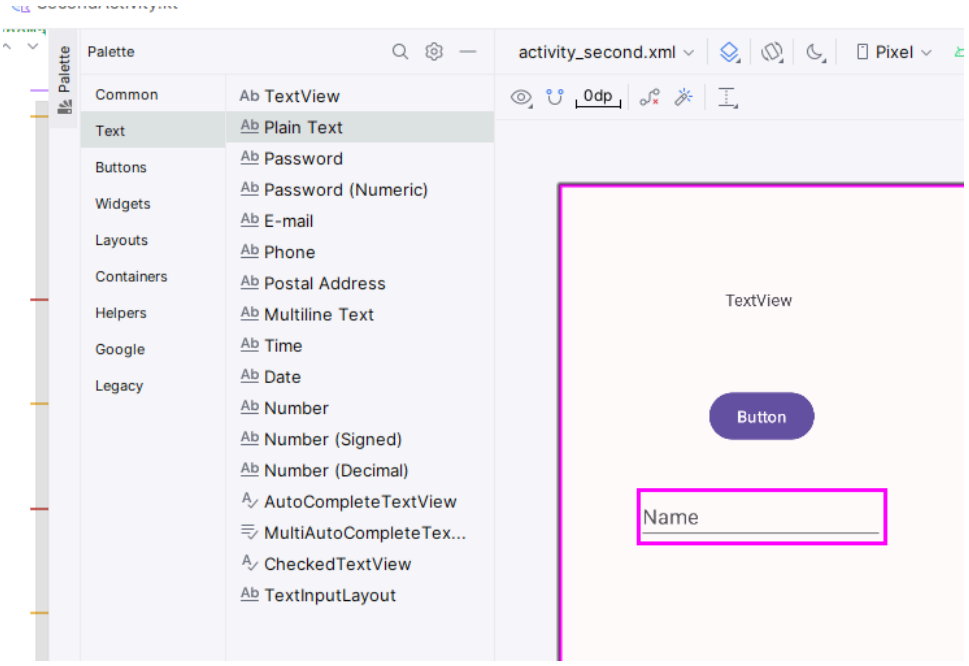
3. Buka `activity_second.xml` dengan mode **Split.** Klik pada tulisan “Palette”, drag komponen `EditText, Button, TextView` ke tempat preview design



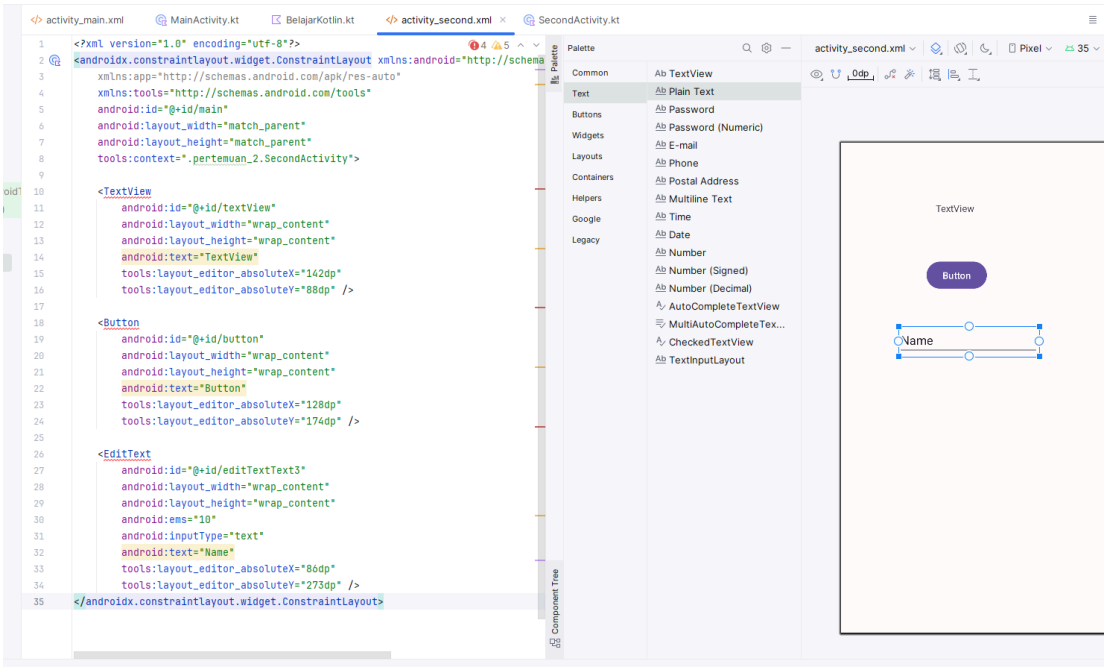
Klik pada bagian Palette untuk menampilkan berbagai macam pilihan komponen



Contoh setelah menambahkan komponen tersebut:



Perhatikan pada bagian *code*, saat menambahkan komponen, secara otomatis IDE akan menambakan kode programnya untuk dapat di sesuaikan lebih lanjut.



4. Beri atribut `android:id=@+id/xxxxxx` pada komponen `TextView`, `Button` dan `EditText`

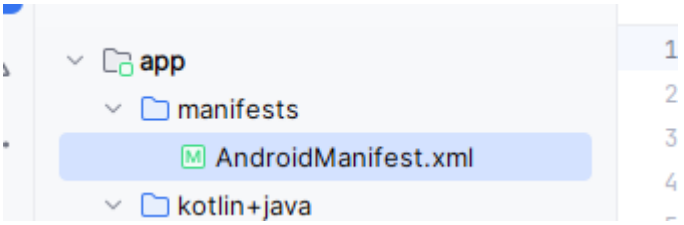
```
<TextView
     android:id="@+id/textNama"
 ....

 <Button
     android:id="@+id/btnSubmit"
 ...

 <EditText
     android:id="@+id/inputNama"
 ....
```

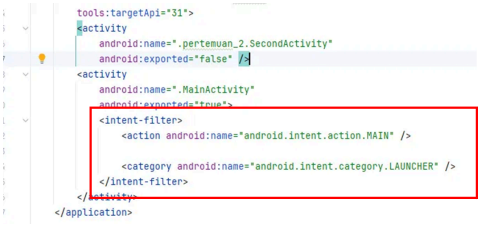
5. Secara default, aplikasi akan membuka `MainActivity`. Agar langsung membuka `SecondActivity` yang baru saja kita buat, lakukan langkah berikut:

    a. Buka `app/manifest/AndroidManifest.xml`



b. Pindahkan `<intent-filter>` ke dalam deklarasi `SecondActivity`.

Before



<intent-filter> awalnya ada di deklarasi MainAcitivty

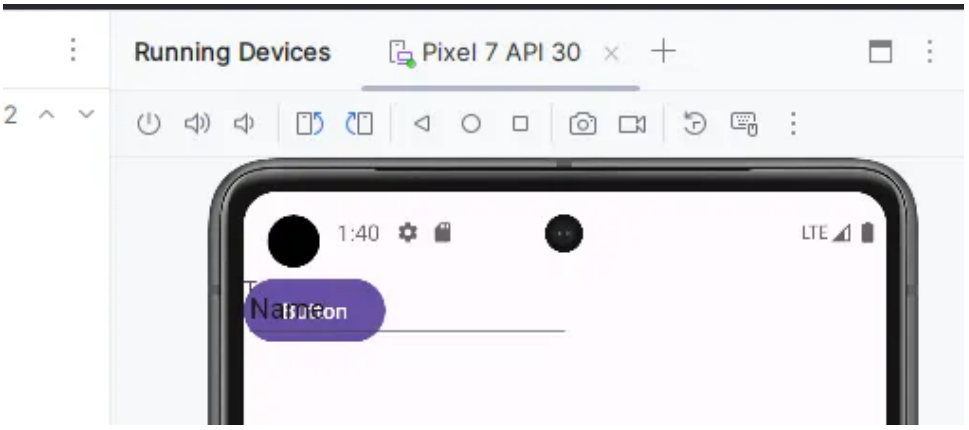
After



Pindahkan ke deklarasi SecondActivity

c. Atur `exported="false"` pada `MainActivity` dan `exported="true"` pada `SecondActivity`

6. Sekarang kita coba *run* dan akan tampil sebagai berikut



Semua komponen akan bertumpuk karena belum menerapkan constraint.

▼ 📂 Penerapan Constraint

> 🔥 Constraint berfungsi untuk menentukan bagaimana sebuah komponen UI diposisikan dan diatur dalam tampilan dimana tampilan tersebut menerapkan layout dengan jenis `ConstraintLayout` .

Silahkan buka `activity_second.xml` , perhatikan bahwa setiap kompoen `TextView` , `Button` , dan `EditText` masih terdapat error/warning yang ditandai dengan garis merah.

```
<TextView
    android:id="@+id/textNama"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="TextView"
    tools:layout_editor_absoluteX="142dp"
    tools:layout_editor_absoluteY="88dp" />

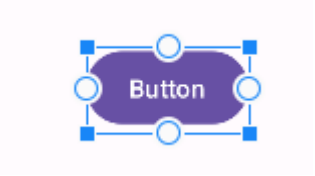
<Button
    android:id="@+id/btnSubmit"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Button" />

<EditText
    android:id="@+id/inputNama"
```

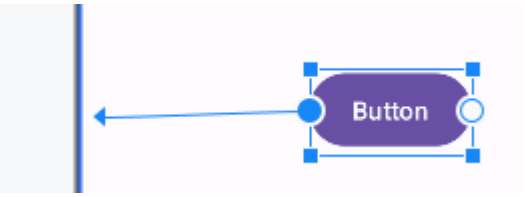
Error tersebut muncul karena belum adanya constraint. Setiap komponen yang ditambahkan pada `ConstraintLayout` , perlu ditetapkan *constraint*-nya atau batas atas/bawah/kiri/kanan komponen agar komponen dapat diam ditempat dan tidak berpindah ke sudut kiri atas layar saat di jalankan.

Ikut langkah berikut untuk menerapkan constraint pada komponen melalui bagian preview:

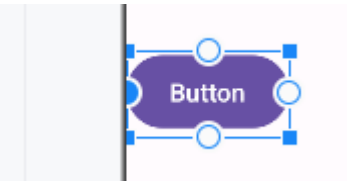
- Klik pada komponen maka akan muncul 4 buah *cirlce* putih yang menandakan belum ada nya *contraint*di ke-4 sisi komponen tersebut (atas-bawah-kiri-kanan)



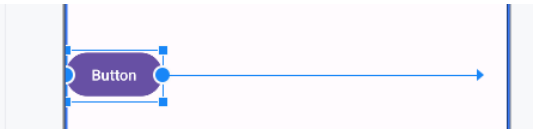
- Klik lingkaran dibagian kiri komponen dan drag ke sisi kiri preview.



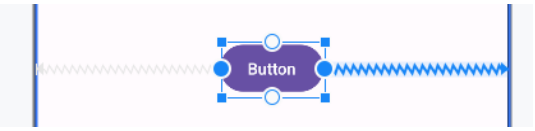
- Maka komponen akan berpindah ke sisi kiri.



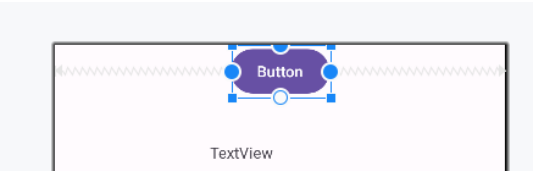
- Lakukan hal yang sama untuk lingkaran bagian kanan, dan drag ke kanan



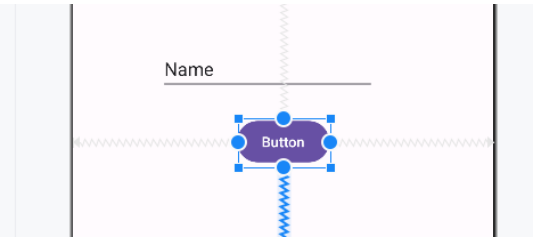
- Maka komponen akan berpindah ke tengah. Ini dikarenakan saat constraint kiri dan kanan di tetapkan, maka artinya komponen akan berada ditengah



- Jika kita menerapkan constraint di bagian atas, maka komponen akan berada diatas layar namun tetap ditengah



- Jika kita terapkan lagi constraint di bagian bawah, maka komponen akan berada tepat di tengah layar



Setiap constraint yang sudah kita terapkan akan muncul juga di bagian kode seperti berikut.

```
<Button
    android:id="@+id/btnSubmit"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Button"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

Perhatikan value dari atribut `app:layout_constraintXXX_XXX` yang berisikan nilai `parent` yang berarti sisi paling luar yang ada pada tampilan tersebut

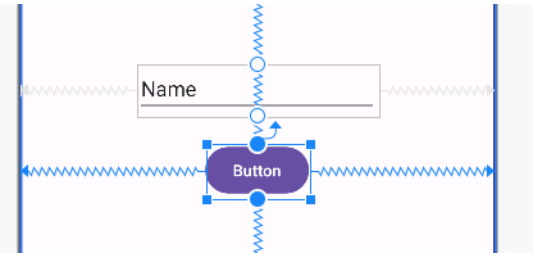
---

Constraint juga dapat diterapkan antar komponen, sehingga kita dapat meletakkan antar komponen sejajar atau berada diatas/dibawah-nya. Contoh:

- Posisikan `inputNama` berada di tengah seperti berikut:

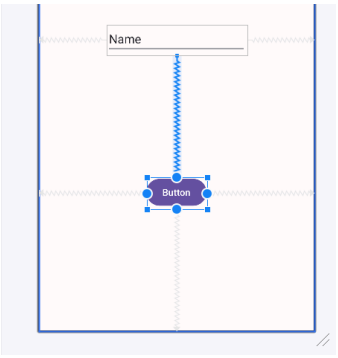


- Lalu untuk `btnSubmit` terapkan constraint atas-nya ke lingkaran pada sisi bawah `inputNama`



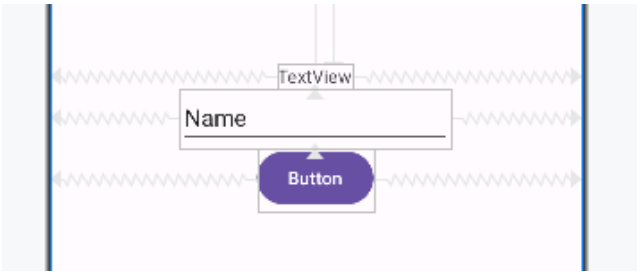
- Maka tombol tersebut akan berada di bawah `inputNama` namun mengambil posisi tengah agak kebawah. Secara kode juga akan berubah dimana yang awalnya `app:layout_constraitnTop_toBottomOf="parent"`

  menjadi `app:layout_constraitnTop_toBottomOf="@id/inputNama`




```
<Button
    android:id="@+id/btnSubmit"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Button"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/inputNama" />
```

Jika kita sudah menerapkan constrain ke semua komponen, kita dapat dengan mudah mengatur letak dan posisi komponen



💡

> Hal yang perlu diperhatikan ialah setiap komponen setidak nya harus memiliki
> 3 constraint agar posisi tidak berubah saat dijalankan

▼ 📂 Event Listener (onClick)

Event Listener merupakan suatu mekanisme yang menangani *event* yang terjadi di aplikasi, seperti klik tombol, sentuhan layar, atau perubahan teks. Beberapa jenis event listener yang dapat diterapkan:

- `setOnLongClickListener` → untuk menangani klik yang ditahan lama.
- `setOnTouchListener` → untuk menangani sentuhan pengguna.
- `setOnFocusChangeListener` → untuk menangani perubahan fokus pada elemen UI.
- `setOnClickListener` → untuk menangani klik pengguna.

🔥 Goal topik ini:

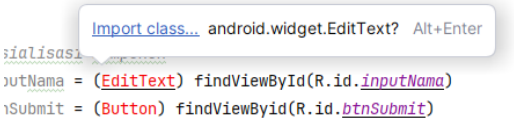
- Menerapkan event onClickListener pada sebuah `Button`
- Mengambil nilai value pada sebuah `EditText` dan menampilkannya pada sebuah `Toast`

Pada `SecondActivity` , kita perlu inisialisasi komponen yang ada pada layout agar dapat diterapkan event atau diambil *value* nya. Silahkan ketikkan kode berikut didalam function `onCreate()`.

```
// Inisialisasi komponen
val inputNama: EditText = findViewById(R.id.inputNama)
```

```
val btnSubmit: Button = findViewById(R.id.btnSubmit)
```

Jika class komponen tersebut belum pernah di-import sebelumnya, akan muncul prompt seperti ini, silahkan klilk “Import Class”



atau import juga dapat dilakukan dengan klik error indikator disebelah kiri yang akan memberikan suggestion untuk melakukan beberapa hal salah satunya ialah “Import Class”



Setelah import berhasil, lanjutkan dengan menambah kode berikut:

```
btnSubmit.setOnClickListener {
    //Mengambil value dari inputNama dan menampilkan di Logcat
  val nama = inputNama.text
    Log.e("Klik btnSubmit","Tombol berhasil di tekan. Isi dari inputNama = $nama")

  Toast.makeText(this, "Anda telah melakukan klik pada tombol Submit", Toast.LENGTH_SHORT).show()
}
```

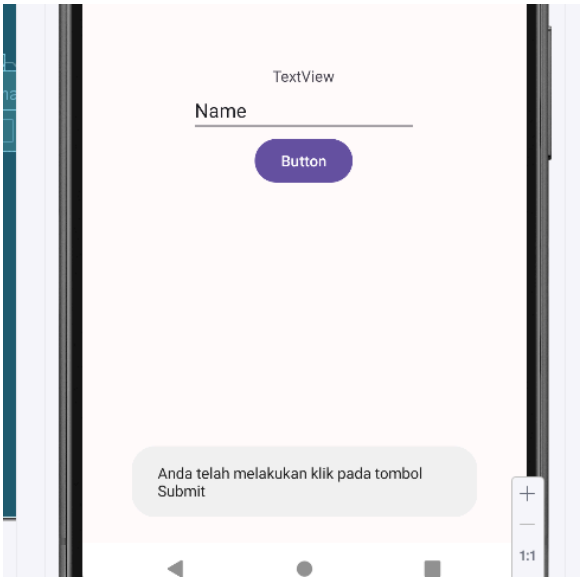
Berikut kode akhir dari `SecondActivity` . Silahkan dijalankan dan klik Button untuk menampilkan `Toast`

```kotlin
class SecondActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_second)
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }

        // Inisialisasi komponen
        val inputNama: EditText = findViewById(R.id.inputNama)
        val btnSubmit: Button = findViewById(R.id.btnSubmit)

        // Event Click pada btnSubmit
        btnSubmit.setOnClickListener {
            val nama = inputNama.text
            Log.i( tag: "Klik btnSubmit", msg: "Tombol berhasil di tekan. Isi dari inputNama = $nama")

            Toast.makeText( context: this,  text: "Anda telah melakukan klik pada tombol Submit", Toast.LENGTH_SHORT).show()
        }
    }
}
```



Informasi Log pada baris 33 juga akan tampil di bagian Logcat seperti berikut:

